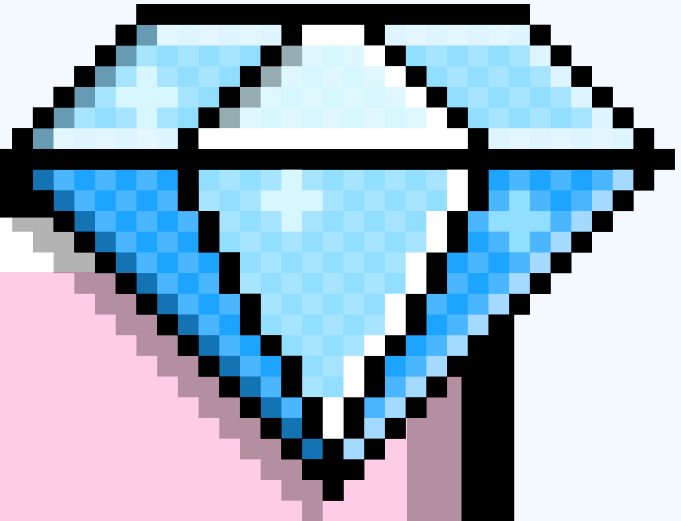
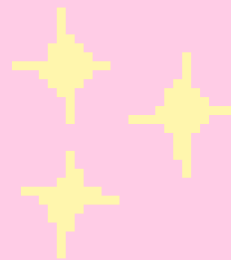
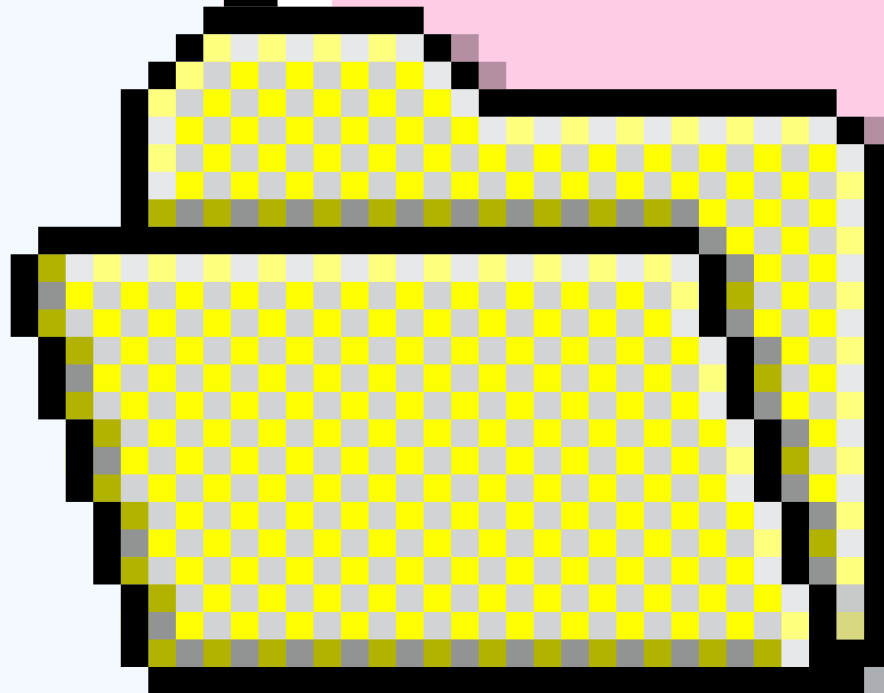




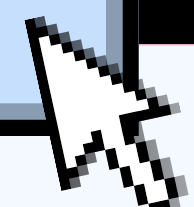
Ambulance Routing System in UPM

Group Members:

- 
1. Aidil Qayyim (223555)
 2. Irfan Daniel (226089)
 3. Aiman Haziq (222707)
 4. Zahin Zulqarnain (222292)
- 



Start



Scenario

EMERGENCY

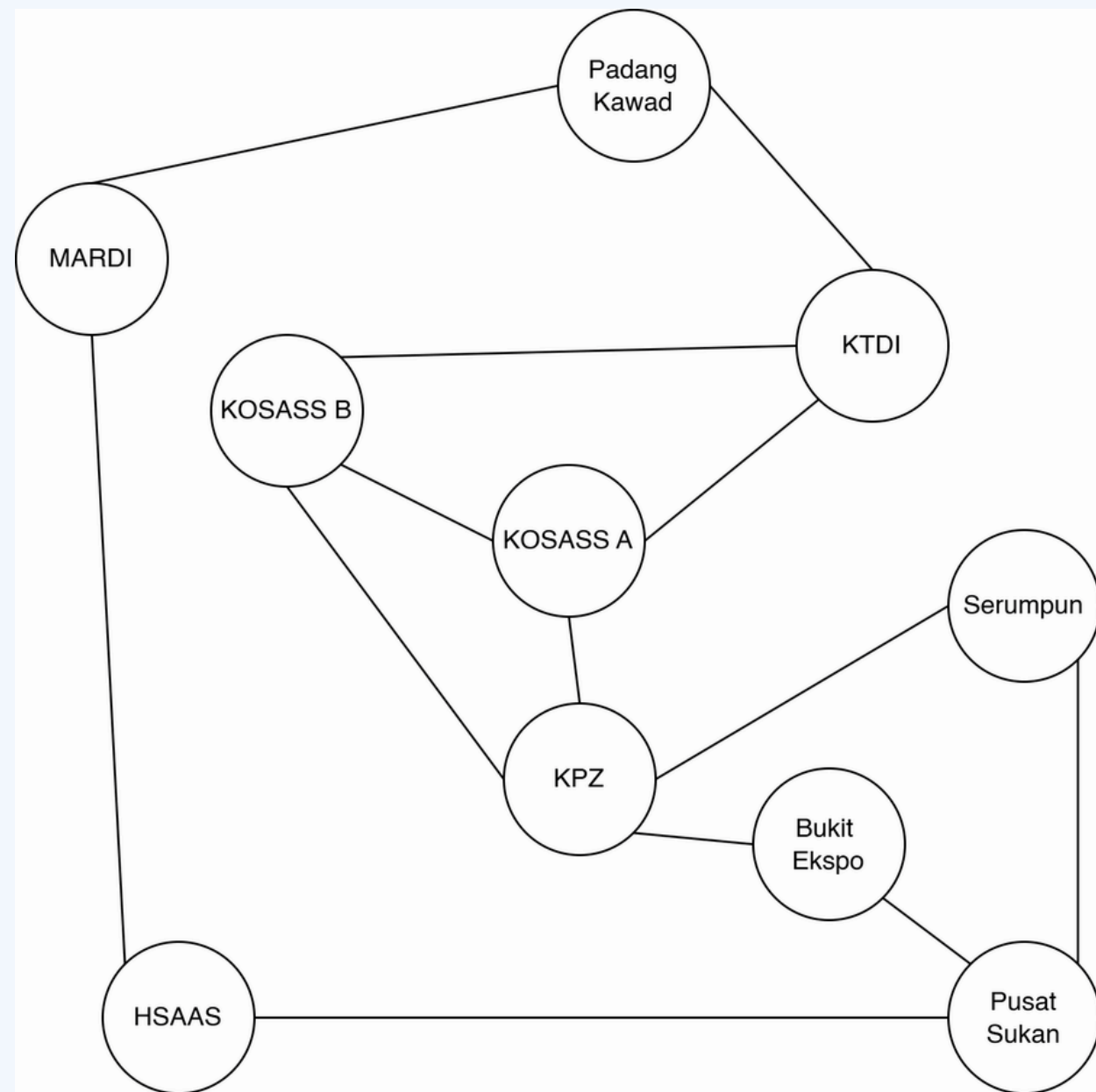
**1:00 PM. WEEKDAY.
UPM CAMPUS.**

**FIRE BREAKS OUT AT KOSASS B — STUDENTS INJURED,
EVACUATION UNDERWAY.
FOOTBALL TOURNAMENT NEAR KTDI — ROADS ARE PACKED
AND CONGESTED.**

ONE AMBULANCE AT HSAAS NEEDS TO REACH KOSASS B.

The Model

THE CAMPUS AS A GRAPH



NODES

- 10 CAMPUS LOCATIONS

EDGES

- 13 CONNECTING ROADS

SOURCE: HSAAS

DESTINATION: KOSASS B



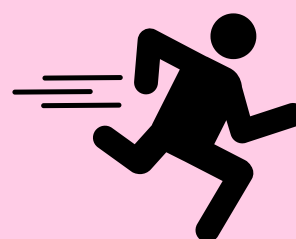
Damage and Impact



The longer the ambulance takes to arrive...



Students affected by smoke inhalation may become worse while waiting for treatment..



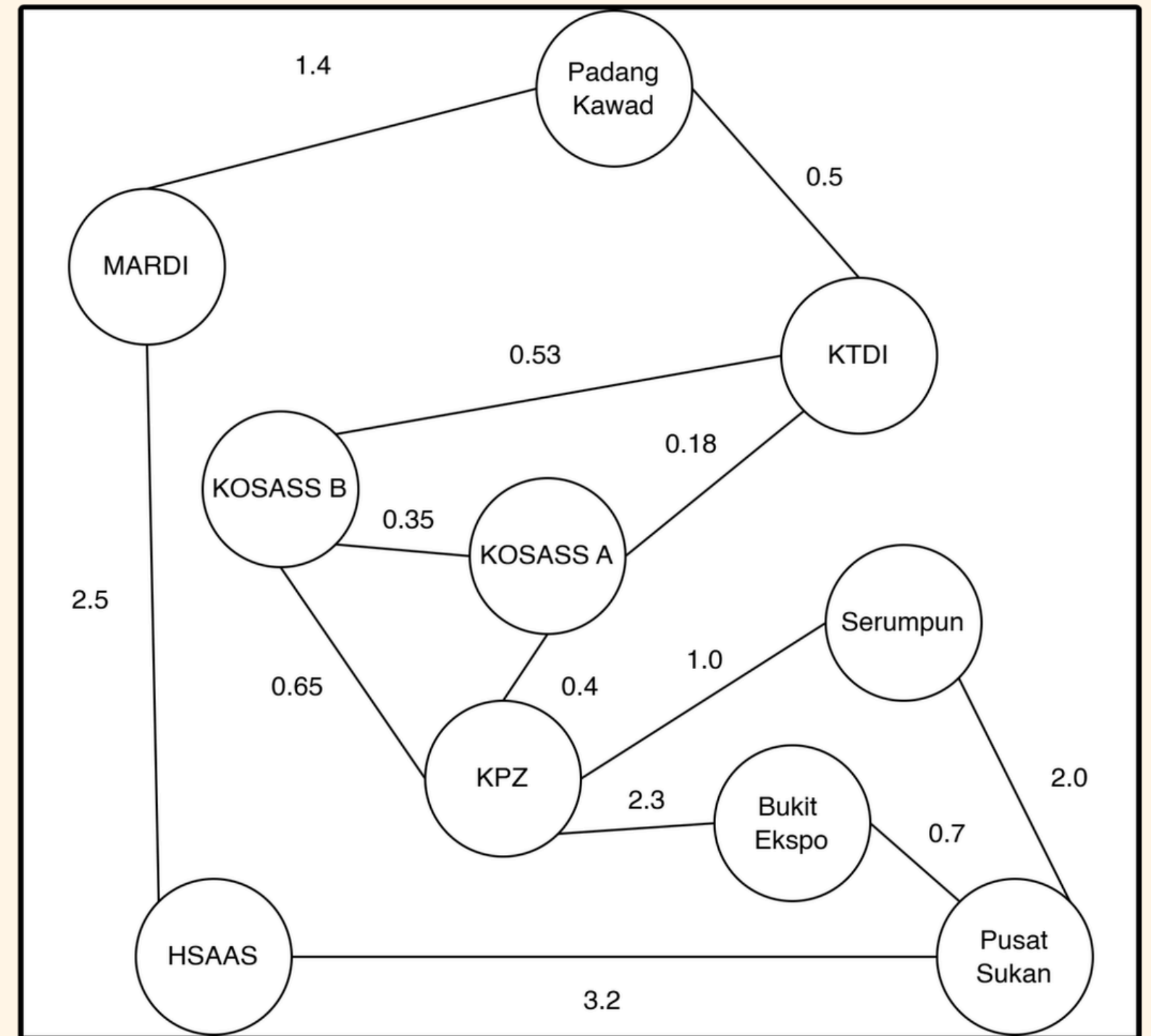
The evacuation process also becomes more difficult because emergency teams are waiting for medical support.



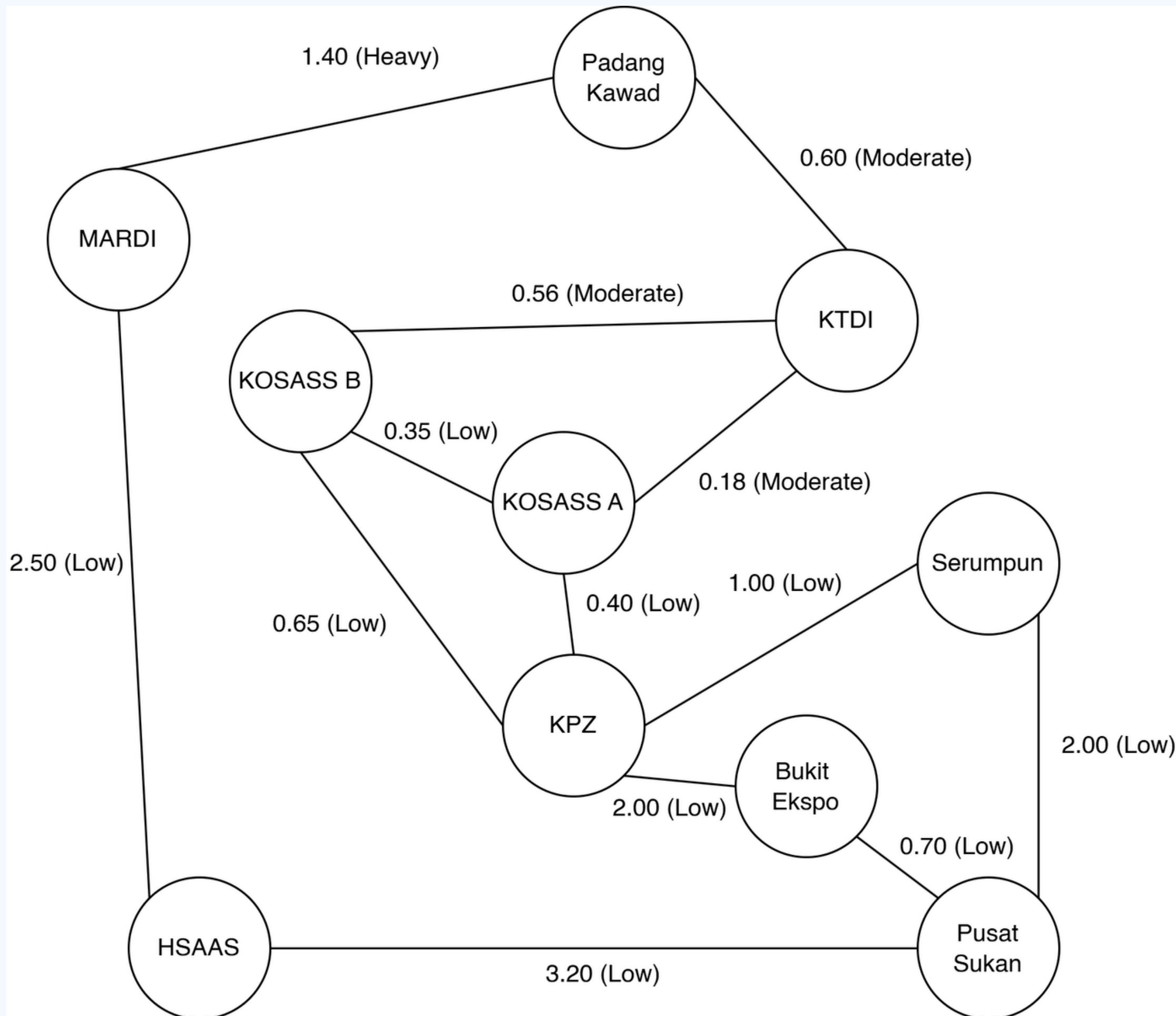
Psychological distress among students increases.

Development of Model

- **Nodes (Vertices)** represent physical locations on the UPM campus. There are 10 nodes in total: HSAAS, KOSASS A, KOSASS B, KPZ, KTDI, MARDI, Padang Kawad, Bukit Ekspo, Serumpun, and Pusat Sukan.
- **Edges** represent roads or paths connecting two locations.
- **Weights** represent the effective travel cost between two connected locations. The cost is calculated by multiplying the road distance (in kilometres) by a traffic multiplier that reflects the severity of traffic congestion.

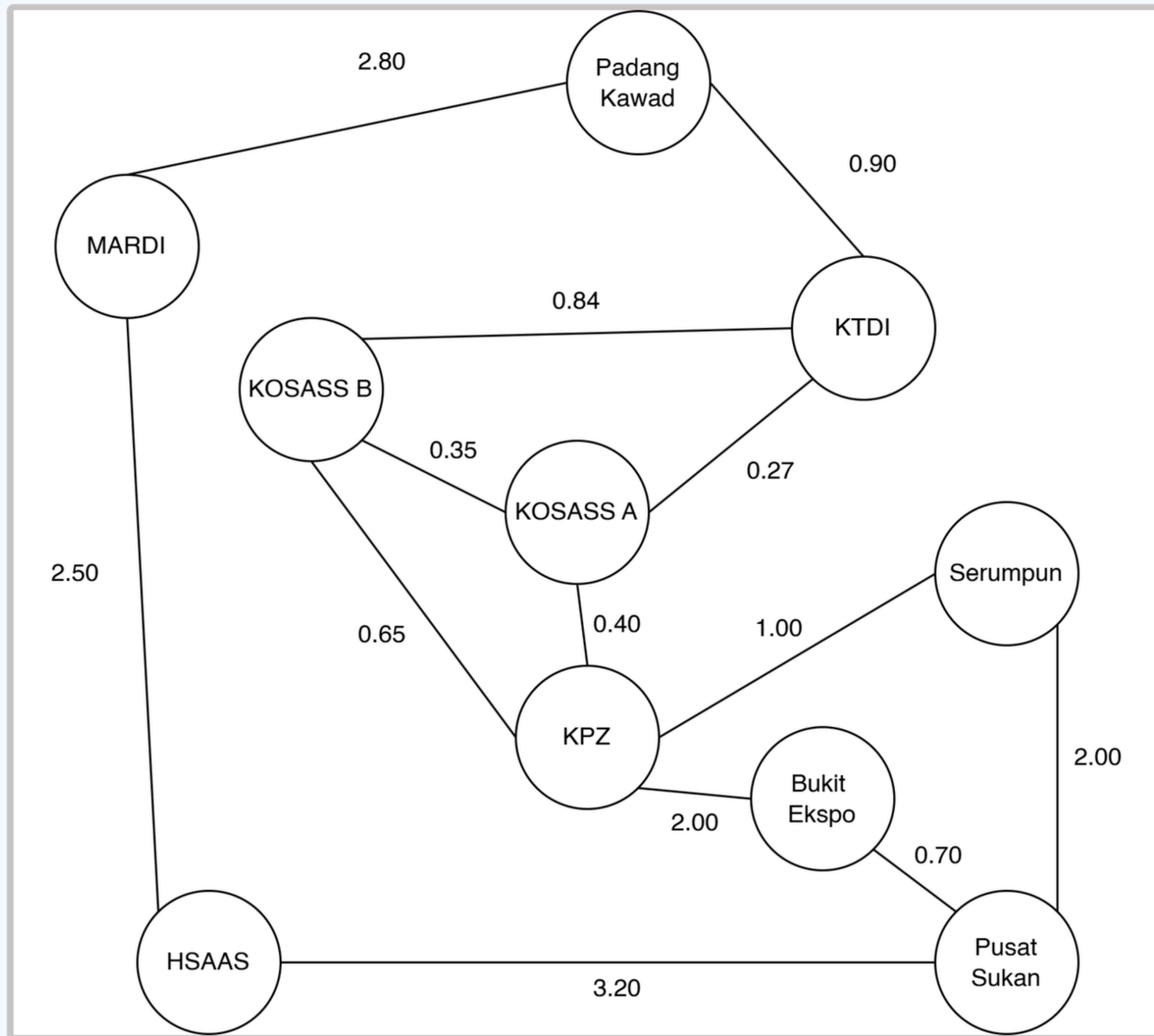


Unfinished graph where the numbers represent the distance in kilometers



- The distance being measured only as weight is not suitable. Traffic level needs to be taken into account as well.
- Roads between MARDI and Padang Kawad are congested during peak hours.
- Any roads leading to or out of KTDI are congested as well because of the football tournament.
- Effective Travel Cost are calculated to better represent the weight rather than only using distance.

Traffic Level	Multiplier
Low	x1.0
Moderate	x1.5
Heavy	x2.0



Model of
the
Chosen
Scenario

Constraints

Graph Constraint

- *Weight must be ≥ 0*
- *The ambulance can only move along existing roads*
- *The graph is undirected and connected*

Scenario Constraint

- *Peak hour traffic increases travel time:*
 - *MARDI $\rightarrow$ Padang Kawad*
 - *HSAAS $\rightarrow$ Pusat Sukan*
- *Football tournament causes extra delays on:*
- *Padang Kawad $\rightarrow$ KTDI*
- *KTDI $\rightarrow$ KOSASS A*
- *KTDI $\rightarrow$ KOSASS B*

Time Constraint

- *Must compute results very quickly*
- *Important for emergency ambulance routing*
- *Even if the graph becomes larger, the algorithm must stay efficient*

Algorithm Constraint

- *Must always find the shortest (optimal) path*
- *Must solve single-source to single-destination routing*

ALGORITHM SELECTION

BFS SEARCH

- Ignores edge weights entirely



FLOYD-WARSHALL

- Computes all pairs — we only need one route



GREEDY BEST-FIRST

- No guarantee of shortest path

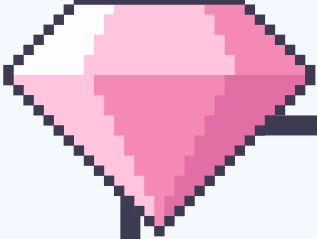


DIJKSTRA'S

- Weights, single source, guaranteed optimal



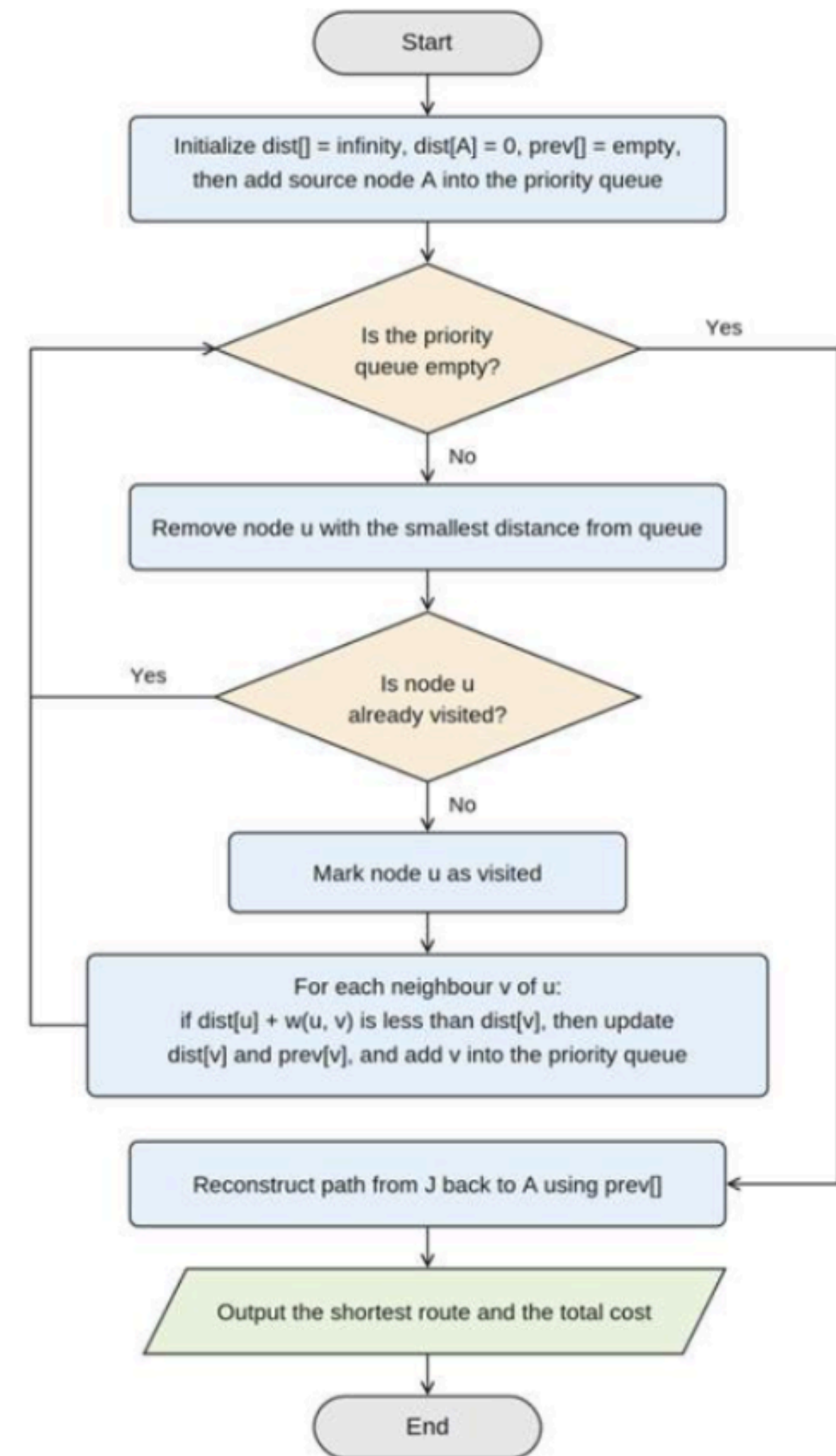
Why Dijkstra's?

- 
- ✓ All edge weights are non-negative — exactly what Dijkstra's needs
 - ✓ Considers actual travel cost, not just number of roads
 - ✓ Solves one route only — no wasted computation
 - ✓ Guarantees the shortest path every single time

The choice is clear !

Flowchart

It starts at **A** with **distance 0** and sets **every other place** to **infinity**. Then it keeps picking the **nearest unvisited place**, **updates** its neighbours if a **shorter road is found**, and repeats until nothing is **left**. Finally it traces the path back from **J to A**.



Pseudocode

The top part sets **all distances to infinity** and the **source to 0**. The **while loop** does the main job. It takes the **nearest node**, then **checks each neighbour** to see if there is a shorter way and updates it. This repeats until the **queue is empty**, then the path is built **back to the start**.

```
ALGORITHM Dijkstra(Graph, source, destination)

INPUT  : Graph G with nodes and weighted edges, a source node and a destination
        node
OUTPUT : the shortest route from source to destination and its total cost

1.  for each node v in G do
2.    dist[v] <- infinity
3.    prev[v] <- undefined
4.  end for
5.  dist[source] <- 0
6.  PQ <- priority queue that holds (distance, node), insert (0, source)
7.
8.  while PQ is not empty do
9.    u <- remove the node with the smallest distance from PQ
10.   if u is already visited then
11.     skip and continue the loop
12.   end if
13.   mark u as visited
14.   for each neighbour v of u do
15.     newDist <- dist[u] + weight(u, v)
16.     if newDist < dist[v] then
17.       dist[v] <- newDist
18.       prev[v] <- u
19.       insert (newDist, v) into PQ
20.     end if
21.   end for
22. end while
23.
24. path <- empty list
25. node <- destination
26. while node is defined do
27.   add node to the front of path
28.   node <- prev[node]
29. end while
30. return path and dist[destination]
```

Program Output

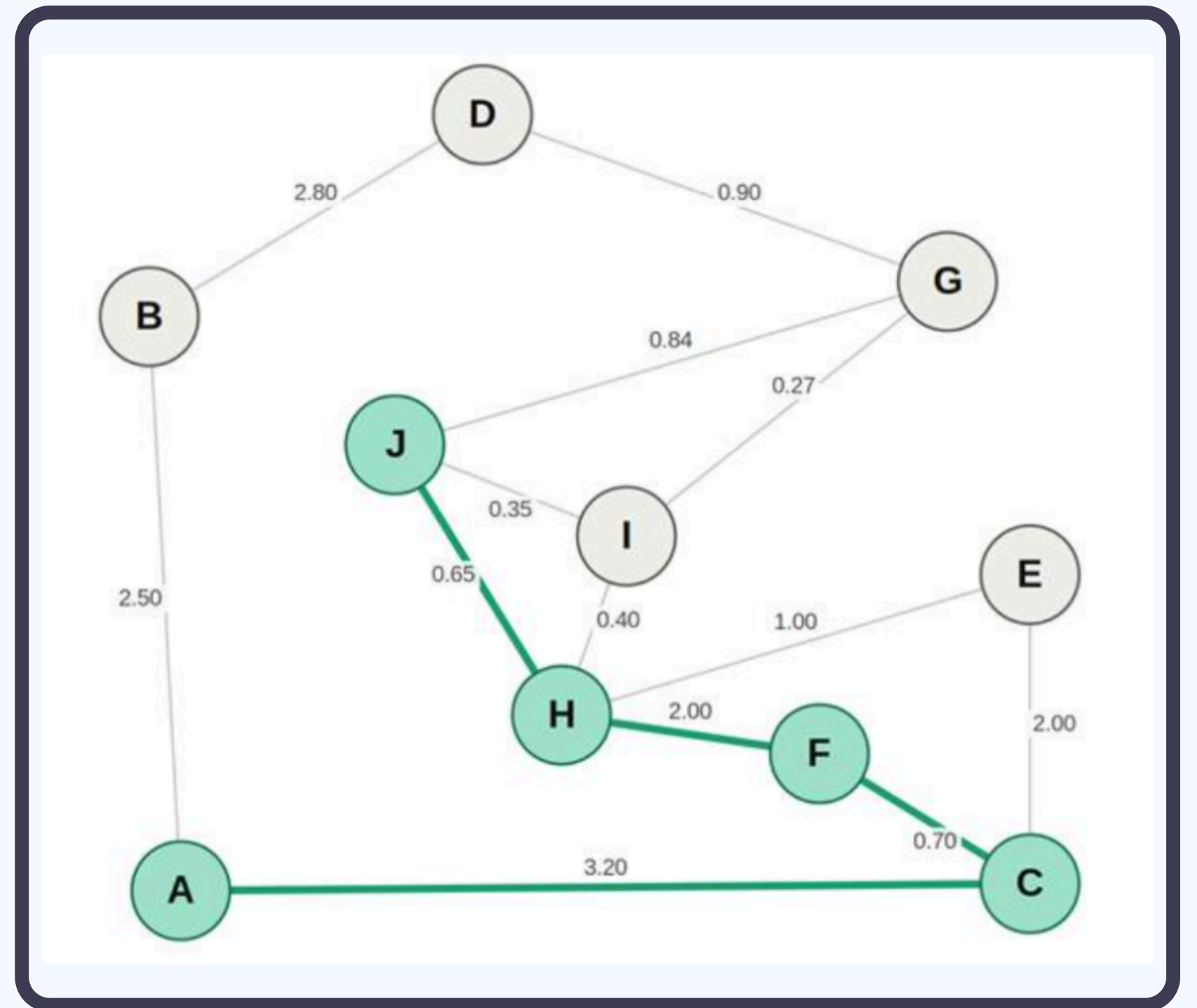
It stores the **campus map as places** and **roads with their costs**, then runs the steps from the flowchart using a **priority queue** that always gives the **nearest place first**.

```
Shortest effective cost from A (HSAAS) to every node
A -> A : 0.00
A -> B : 2.50
A -> C : 3.20
A -> D : 5.30
A -> E : 5.20
A -> F : 3.90
A -> G : 6.20
A -> H : 5.90
A -> I : 6.30
A -> J : 6.55

Source node      : A (HSAAS)
Destination node : J (KOSASS B)
Optimal route    : A -> C -> F -> H -> J
Total cost       : 6.55

Time taken       : 2386300 nanoseconds
                  : 2.3863 milliseconds
```


Suggested Route from Output



ALGORITHM VERIFICATION

CORRECTNESS VERIFICATION

- Manually trace Dijkstra's Algorithm step-by-step and compare the results with the program output.

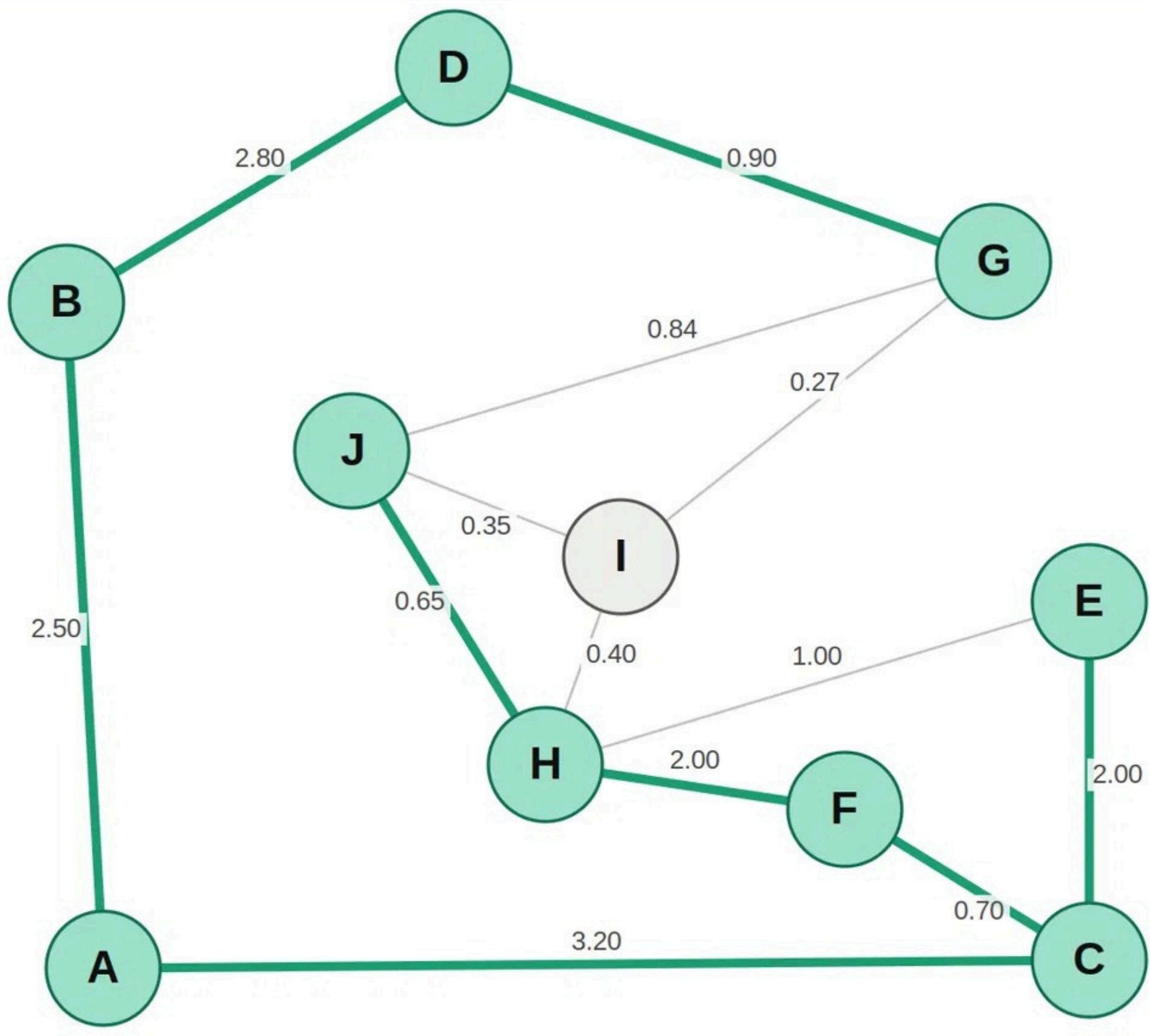
RECURRENCE ANALYSIS

- Applied the formula $d[v] = \min(d[v], d[u] + w(u,v))$ to update the shortest known distance for each node.

ASYMPTOTIC VERIFICATION

- Verified that the implementation follows Dijkstra's theoretical time complexity of $O((V + E) \log V)$.

Correctness Verification



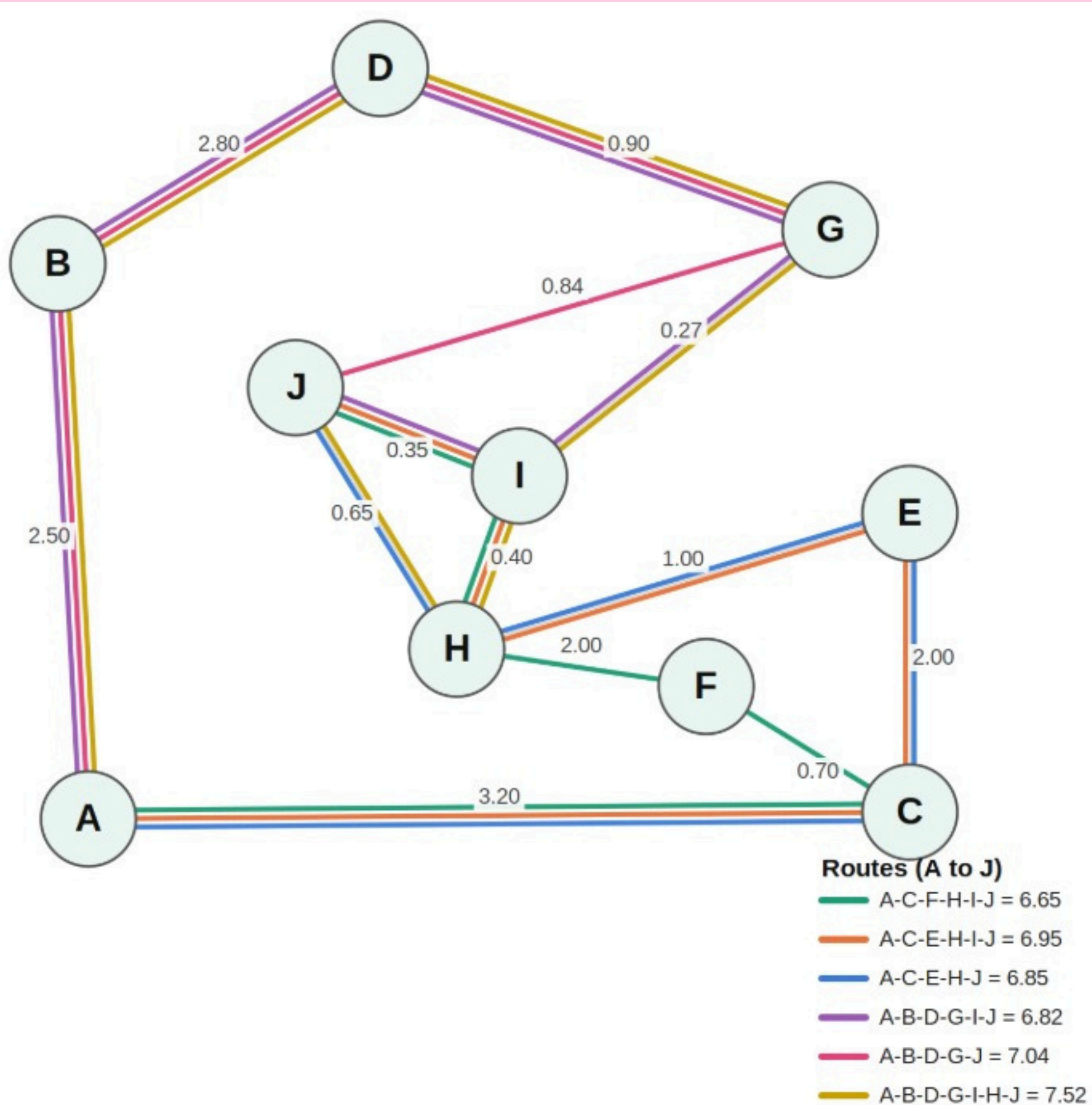
Visited Nodes = {A, B, C, F, E, D, H, G, J}

Q:	A	B	C	D	E	F	G	H	I	J
	0									
A		2.50	3.20							
B			3.20	5.30						
C				5.30	5.20	3.90				
F				5.30	5.20			5.90		
E				5.30				5.90		
D							6.20	5.90		
H							6.20		6.30	6.55
G									6.30	6.55
J										6.55

- A → C → F → H → J with a total cost of 6.55
- Manual calculation is the same as program output

Recurrence Analysis

- Applied the relaxation formula $d[v] = \min(d[v], d[u] + w(u,v))$ repeatedly to compare current distances with newly discovered paths.
- The algorithm will also evaluate different routes to the destination node
- The best route to J is:
 - $A \rightarrow C \rightarrow F \rightarrow H \rightarrow J$
 - Total cost = $5.90 + 0.65 = 6.55$
- Other possible routes are:
 - $A \rightarrow C \rightarrow F \rightarrow H \rightarrow I \rightarrow J$ (Total cost = 6.65)
 - $A \rightarrow C \rightarrow E \rightarrow H \rightarrow I \rightarrow J$ (Total cost = 6.95)
 - $A \rightarrow C \rightarrow E \rightarrow H \rightarrow J$ (Total cost = 6.85)
 - $A \rightarrow B \rightarrow D \rightarrow G \rightarrow I \rightarrow J$ (Total cost = 6.82)
 - $A \rightarrow B \rightarrow D \rightarrow G \rightarrow J$ (Total cost = 7.04)
 - $A \rightarrow B \rightarrow D \rightarrow G \rightarrow I \rightarrow H \rightarrow J$ (Total cost = 7.52)



Asymptotic Verification

Dijkstra’s Algorithm time complexity is:

$$O((V + E) \log V)$$

Based on the scenario:

- Number of vertices, $V = 10$
- Number of edges, $E = 13$

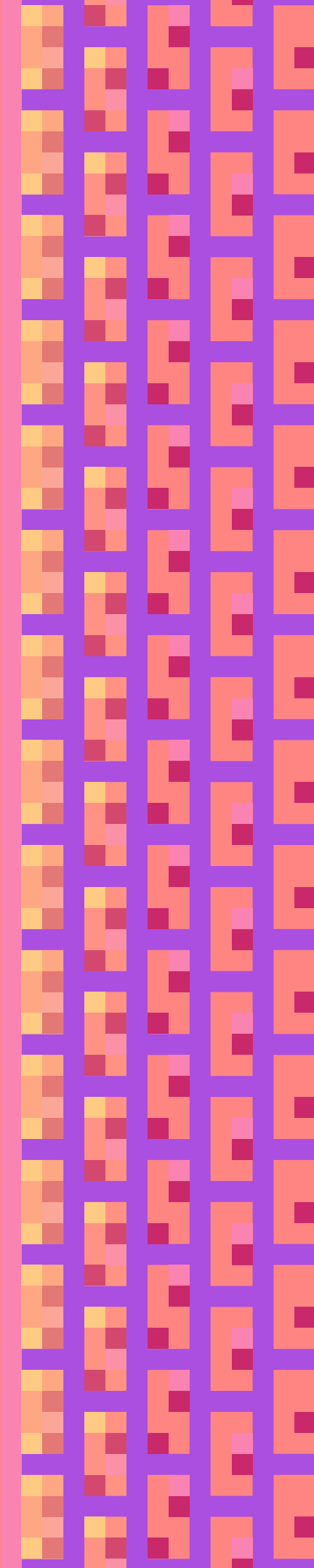
Substituting these values into the complexity formula:

$$\begin{aligned} &= O((10+13) \log_2 10) \\ &\approx O(23 \times 3.32) \\ &\approx O(76.4) \end{aligned}$$

What if expand the campus road network?

Graph Size	Approximate Operations
$V=10, E=13$ (current)	~76
$V=50, E=80$	~693
$V=100, E=200$	~1,993
$V=1000, E=3000$	~39,900

- The number of operations increase gradually, which it should.
- Compared to Floyd-Warshall Algorithm, Dijkstra’s Algorithm need lesser number of operations.
- For example, when $V = 1000$, Dijkstra's Algorithm requires only about 39,900 operations
- While Floyd-Warshall Algorithm which has time complexity of V^3 , it require approximately 1,000,000,000 operations



Analysis of the Algorithm

To measure performance of algorithm.

TIME COMPLEXITY

The amount of time taken by an algorithm to run as a function of the length of the input.

SPACE COMPLEXITY

The amount of memory required by the algorithm to solve given problem.

TIME COMPLEXITY

BEST CASE: $O((V+E) \log V)$

Happens when the graph is sparse, meaning the number of edges is close to the number of vertices.

WORST CASE: $O(V^2 \log V)$

occurs when the graph is dense, with many edges, and the priority queue operations become less efficient due to the lack of optimization.

AVERAGE CASE: $O((V+E) \log V)$

Same complexity as “Best Case”. Each vertex is processed once, and each edge may trigger at most one priority queue update.

SPACE COMPLEXITY

Total Space Complexity:
 $O(V+E)$

Example: $V = 10, E = 13$
 $O(V + E) = O(23)$

Data Structures	Spaced Used
Adjacency list	$O(V+E)$
Distance Array	$O(V)$
Previous Array	$O(V)$
Visited Array	$O(V)$
Priority Queue	$O(E)$ at most
Adjacency list	$O(V+E)$
Total	$O(V+E)$

CONCLUSION

In conclusion, Dijkstra's Algorithm successfully determined the optimal ambulance route from HSAAS to KOSASS B with a total cost of 6.55. The algorithm was proven to be correct, efficient, and suitable for weighted road networks with traffic conditions. With a time complexity of $O((V + E) \log V)$ and space complexity of $O(V + E)$, it provides a practical solution for emergency route planning and can be scaled to larger transportation networks in the future.

Thank
you

